

Universidade São Judas Tadeu

Andressa Emily Rabêlo Pereira 823213904
Júlia Oliveira Rocha 823214680
Lucas Marzocca 823116813
Matheus Henrique Ferreira Guimarães 823141914
Marcos Vinicius Santos 82327399
Mylene Soares Rocha 824144075

Projeto Meowlang — Analisador ILéxico

Teoria da Computação e Compiladores

Profª Milkes Yone Alvarenga

**São Paulo
2026**

Índice

1. DESCRIÇÃO DO PROJETO.....	2
2. FUNDAMENTAÇÃO TEÓRICA.....	2
2.1 Gramática da Linguagem MeowLang	2
2.2 Categorias de Tokens	3
3. ETAPAS DA ANÁLISE LÉXICA	4
3.1 Leitura do Código-Fonte	4
3.2 Eliminação de Espaços e Quebras de Linha	4
3.3 Remoção de Comentários	4
3.4 Reconhecimento de Tokens	4
4. VERIFICAÇÃO SINTÁTICA	5
4.1 Verificações Realizadas.....	5
4.2 Comportamento em Caso de Erro	5
5. INTERPRETADOR DE COMANDOS	5
5.1 Funcionalidades do Interpretador	5
5.2 Verificação de Tipos.....	6
6. IMPLEMENTAÇÃO DO SISTEMA	6
6.1 Arquitetura do Código	6
7. FUNCIONALIDADES DA INTERFACE.....	7
8. EXEMPLOS DE CÓDIGO MEOWLANG.....	7
8.1 Declaração de Variáveis	7
8.2 Estrutura Condicional.....	7
8.3 Laço de Repetição	7
9. CONCLUSÃO	8

1. DESCRIÇÃO DO PROJETO

O projeto MeowLang consiste na implementação de um analisador léxico para uma linguagem de programação esotérica de temática felina, desenvolvido como trabalho prático da disciplina de Linguagens Formais e Autômatos.

O sistema realiza as etapas de tokenização e verificação sintática de código-fonte escrito em MeowLang, identificando e classificando cada elemento da linguagem.

Além da engine de análise, o projeto apresenta uma interface interativa em ambiente web, construída com HTML5, CSS3 e JavaScript, que permite escrever, executar e visualizar a análise do código em tempo real.

Entre os recursos da interface estão:

- Editor de código com numeração de linhas;
- Exemplos pré-carregados de código meowlang;
- Visualização dos tokens identificados em badges coloridos;
- Tabela de variáveis declaradas e seus tipos;
- Console de saída dos comandos executados;
- Detecção e exibição de erros léxicos e sintáticos;
- Log animado de compilação.

O objetivo principal é demonstrar, de forma prática e visual, como um analisador léxico processa o código-fonte, separando-o em tokens classificados, e como a verificação sintática identifica erros estruturais na linguagem.

2. FUNDAMENTAÇÃO TEÓRICA

A análise léxica é a primeira fase de um compilador ou interpretador. Seu objetivo é ler o código-fonte caractere a caractere e agrupá-los em unidades significativas denominadas tokens.

Formalmente, um **token** é um par:

⟨tipo, valor⟩

Onde:

- **Tipo** identifica a categoria do token (palavra-chave, identificador, operador, etc.)
- **Valor** é o lexema, ou seja, a sequência de caracteres reconhecida

2.1 Gramática da Linguagem MeowLang

A MeowLang é uma linguagem esotérica cujos elementos léxicos utilizam termos relacionados ao universo felino. A tabela abaixo apresenta os principais construtores da linguagem:

Palavra-chave	Equivalente	Descrição
psiu	var	variável genérica
psiu_int	int	inteiro
psiu_float	float	número decimal
psiu_string	string	texto
psiu_bool	bool	booleano
psiu_lista	array	lista / vetor
ronrom	true	verdadeiro
bufar	false	falso
dormindo	null	nulo
se_miar	if	condicional
hiss	else	alternativa
cacar	while	laço de repetição condicional
cada_pelo	for	laço de repetição com contador
miar	print	imprimir na saída
arranhar	return	retornar valor
ronronar	function	declaração de função
gatil	class	declaração de classe

2.2 Categorias de Tokens

O analisador léxico classifica cada token reconhecido em uma das seguintes categorias:

Tipo	Descrição	Exemplos
KEYWORD	palavras reservadas da linguagem	<i>psiu_int, se_miar, miar</i>
BOOLEAN	literais booleanos	<i>ronrom, bufar</i>
IDENTIFICADOR	nomes de variáveis e funções	<i>nome, contador, resultado</i>
STRING	literais de texto	<i>"Olá gato"</i>
NUMERO	literais numéricos inteiros e decimais	<i>42, 3.14, 0</i>
OPERADOR	operadores aritméticos e relacionais	<i>+ - * / == != <= >=</i>

DELIMITADOR	pontuação estrutural	<code>() {} [] , ; .</code>
COMENTARIO	texto ignorado pelo compilador	<code># linha, // linha, /* bloco */</code>
ERRO	token inválido não reconhecido	<code>@, \$, @@invalid</code>

3. ETAPAS DA ANÁLISE LÉXICA

O processo de análise léxica do MeowLang é realizado em etapas sequenciais, conforme descrito a seguir.

3.1 Leitura do Código-Fonte

O código-fonte é lido caractere a caractere. A cada iteração, o analisador identifica o padrão que o caractere atual inicia e determina qual regra de reconhecimento aplicar.

3.2 Eliminação de Espaços e Quebras de Linha

Espaços, tabulações e quebras de linha são removidos do fluxo de tokens. Esses elementos são registrados separadamente na categoria de tokens eliminados, permitindo que a interface exiba estatísticas sobre o código processado.

3.3 Remoção de Comentários

A linguagem MeowLang suporta três formas de comentários:

- Comentário de linha iniciado por `#`
- Comentário de linha iniciado por `//`
- Comentário de bloco delimitado por `/*` e `*/`

Todos os comentários são identificados antes dos demais tokens e removidos do fluxo principal, sendo registrados na lista de tokens eliminados.

3.4 Reconhecimento de Tokens

Após a remoção de espaços e comentários, o analisador reconhece os tokens na seguinte ordem de prioridade:

- Strings delimitadas por aspas simples ou duplas
- Números inteiros e decimais
- Operadores duplos (`==`, `!=`, `<=`, `>=`, `&&`, `||`)
- Operadores simples (`+`, `-`, `*`, `/`, `=`, `<`, `>`, `!`)
- Delimitadores `() {} [] , ; .`
- Palavras — verificadas como palavra-chave, booleano ou identificador
- Qualquer sequência não reconhecida é marcada como erro léxico

4. VERIFICAÇÃO SINTÁTICA

Além da análise léxica, o sistema realiza uma verificação sintática básica sobre o código-fonte, checando a integridade estrutural do programa antes de executar os comandos.

4.1 Verificações Realizadas

Verificação	Descrição
Balanceamento de { }	todo bloco aberto com { deve ser fechado com }
Balanceamento de ()	todo parêntese aberto deve ser fechado
Strings fechadas	strings abertas com aspas devem ser fechadas na mesma linha ou bloco

4.2 Comportamento em Caso de Erro

Quando erros são detectados, o sistema interrompe a execução e exibe uma tela de depuração com a mensagem **"Errou e Molhou o Gato"**, indicando o tipo de erro (léxico ou sintático), a linha onde ocorreu e uma descrição detalhada do problema.

Erros léxicos (tokens inválidos) são priorizados sobre erros sintáticos. Somente após a ausência de erros léxicos o sistema realiza a verificação sintática.

5. INTERPRETADOR DE COMANDOS

Após as etapas de análise léxica e verificação sintática, o sistema executa um interpretador de comandos que percorre os tokens classificados e executa as instruções da linguagem MeowLang.

5.1 Funcionalidades do Interpretador

Recurso	Descrição
Declaração de variáveis	suporta tipagem explícita com verificação de tipo em tempo de execução
Impressão — miar	exibe valores e concatenações de strings no console de saída
Condicional — se_miar / hiss	avalia expressões booleanas e executa o bloco correspondente
Laço while — cacar	repete o bloco enquanto a condição for verdadeira (limite de 20 iterações)
Laço for — cada_pelo	executa laço com inicialização, condição e incremento (limite de 50 iterações)
Retorno — arranhar	retorna e exibe um valor no console de saída

Expressões aritméticas	avalia operações de soma, subtração, multiplicação e divisão entre variáveis e literais
-------------------------------	---

5.2 Verificação de Tipos

O interpretador realiza verificação de tipos em tempo de execução para as declarações tipadas da linguagem:

- **Psiu_int** aceita apenas valores inteiros;
- **Psiu_float** aceita apenas valores decimais;
- **Psiu_bool** aceita apenas `ronrom` ou `bufar`;
- **Psiu_string** converte qualquer valor para texto;
- **Psiu_lista** inicializa uma lista vazia quando o valor não for um array.

6. IMPLEMENTAÇÃO DO SISTEMA

O sistema foi desenvolvido utilizando tecnologias web padrão, sem dependências externas:

Tecnologia	Função
HTML5	estrutura da interface e organização dos painéis
CSS3	estilização, animações e tema visual escuro (dark mode)
JavaScript	tokenizador, verificador sintático e interpretador de comandos
Google Fonts	tipografia: VT323 (estilo terminal), Inconsolata (monospace), Syne (títulos)

6.1 Arquitetura do Código

O código JavaScript é organizado nos seguintes módulos:

- **Tokenize(src)** — realiza a análise léxica e retorna tokens, tokens removidos e erros;
- **Checksyntax(src)** — realiza a verificação sintática e retorna os erros encontrados;
- **Interpretcommands(tokens, vars)** — interpreta os tokens e executa os comandos;
- **Funções de renderização** — constroem o html da saída a partir dos resultados;
- **Funções de interface** — gerenciam abas, editor, exemplos e o log de compilação.

7. FUNCIONALIDADES DA INTERFACE

Funcionalidade	Descrição
Editor com numeração	editor de código com gutter de numeração de linhas atualizado dinamicamente
Exemplos pré-carregados	7 exemplos de código: básico, tipos, if/else, while, for, erro léxico e erro sintático
Painel de gramática	exibe todas as palavras-chave organizadas em abas por categoria
Badges de tokens	cada token é exibido com cor e rótulo específico por categoria
Log de compilação	animação de 14 etapas simulando o processo de compilação (aproximadamente 8 segundos)
Console de saída	exibe os resultados da execução dos comandos miar, arranhar e estruturas de controle
Tabela de variáveis	lista todas as variáveis declaradas com nome, tipo inferido e valor atual
Estatísticas	exibe contagem de tokens, palavras-chave, identificadores, tokens removidos e erros
Tela de erro animada	exibe mensagem "Errou e Molhou o Gato" com detalhes de depuração léxica ou sintática

8. EXEMPLOS DE CÓDIGO MEOWLANG

8.1 Declaração de Variáveis

Exemplo de declaração de variáveis com tipos explícitos:

```
psiu_string nome    = "Garfield"
psiu_int    idade    = 3
psiu_float  peso     = 4.750
psiu_bool   ativo    = ronrom
```

8.2 Estrutura Condicional

```
se_miar (fome > 5) {
    miar("Estou com fome!")
} hiss {
    miar("Satisfeito!")
}
```

8.3 Laço de Repetição

```
cada_pelo (psiu_int i = 0; i < 5; psiu_int i = i + 1) {
    miar("Iteracao: " + i)
}
```

9. CONCLUSÃO

O desenvolvimento do projeto MeowLang permitiu aplicar na prática os conceitos estudados na disciplina de Linguagens Formais e Autômatos, especialmente a construção de um analisador léxico capaz de processar e classificar os elementos de uma linguagem formal.

A partir da definição de uma gramática própria, com palavras-chave inspiradas no universo felino, foi possível implementar todas as etapas do processo de análise léxica: leitura do código-fonte, eliminação de espaços e comentários, reconhecimento de padrões e classificação dos tokens. A implementação evidenciou como as expressões regulares e os autômatos finitos são a base formal que sustenta os analisadores léxicos utilizados em compiladores reais.

Além da análise léxica, o projeto incluiu uma verificação sintática básica e um interpretador de comandos, tornando possível a execução de programas completos escritos em MeowLang, com suporte a variáveis tipadas, estruturas condicionais, laços de repetição e operações aritméticas.

A interface interativa desenvolvida em HTML5, CSS3 e JavaScript permitiu visualizar de forma clara e didática cada etapa do processo, desde a tokenização até a exibição do console de saída e da tabela de variáveis, aproximando a teoria dos autômatos de uma aplicação prática e acessível.

O projeto demonstrou que os modelos formais estudados na disciplina não se limitam ao campo teórico, sendo fundamentais para o desenvolvimento de ferramentas reais de processamento de linguagens. A experiência consolidou o entendimento sobre a importância da análise léxica como primeira etapa de qualquer compilador ou interpretador, e evidenciou como conceitos como autômatos, expressões regulares e gramáticas formais estruturam o funcionamento dos sistemas computacionais modernos.